BTC/USDT Funding Rate Arbitrage

Project Summary

This project investigates whether funding rate signals in BTC/USDT perpetual futures contain exploitable carry alpha, and if so, under what market microstructure conditions that alpha is best captured. The research pipeline spans data acquisition, feature engineering, carry alpha analysis, adverse selection estimation, strategy optimisation, and backtesting — built end-to-end in Python using Polars and NumPy, divided into modules.

The data pulled is in 1 hour increments, the reason being that the alpha decay half life is roughly 50 days, a substantial length of time. However the trade off in using a longer time increment in the data is that we sacrifice some execution modelling realism. Since most of the pnl of the strategy is concentrated in the funding rate payments over a long period of time and any finer data introduces large amounts of noise, for the purposes of this project I have judged the negative impact of using 1 hour over more fine data negligible.

Why does this opportunity exist? The funding rate mechanism is an economic incentive introduced by the exchange to incentivise traders to keep the prices of the perpetual future and spot close together, since the perp never expires so there is no expiry to force the prices to converge and eliminate an arbitrage. The funding rate mechanism works thus: every 8 hours a funding rate is calculated by:

$$F = \begin{cases} P - 0.05\%, & \text{if } I - P < -0.05\%, \\ I, & \text{if } -0.05\% \leq I - P \leq 0.05\%, \\ P + 0.05\%, & \text{if } I - P > 0.05\%. \end{cases}$$

where

- F is the **funding rate** applied during the funding interval.
- P is the **premium index**, which measures the percentage difference between the perpetual futures price and the underlying spot index price.
- I is the **interest rate component**, which on Binance is typically fixed at 0.01% per 8-hour funding period for BTCUSDT contracts.

When the perpetual contract trades above spot ($P > 0$), longs generally pay shorts; when it trades below spot ($P < 0$), shorts generally pay longs. This funding transfer creates an economic incentive for traders to take positions that push the perpetual price back toward the spot price.

The funding payment exchanged at each funding timestamp is

$$\text{Funding Payment} = Q \times M \times F,$$

where

- Q is the position size (in BTC),
- M is the mark price of Bitcoin,
- F is the funding rate for the funding interval.

Equivalently, if $N = Q \times M$ denotes the notional value of the position,

$$\text{Funding Payment} = N \times F.$$

The core funding rate arbitrage strategy is a delta-neutral funding arb: when the perpetual funding rate substantially exceeds zero, short the perp and long an equivalent spot position - and vice versa.

Of course there are costs and risks to account for which include: fees, slippage during execution (due to moving the market and crossing the spread during execution), lack of liquidity, leverage risks and tail events.

1 Problem Formulation

The main question I want to answer is whether there is an exploitable alpha, and how long this alpha lasts. This is the purpose of the carry analysis module. Then I want to account for adverse selection and slippage effects by estimating impact beta. This empirical impact beta is then used in my strategy optimizer module, which aims to see where the Sharpe is highest over a range of combinations of holding period and entry signal, accounting for execution costs. After identifying the best parameters over 6 months of data, this strategy is then tested on the next 6 months of data to see if the Sharpe holds up under out of sample testing.

2 Data Pipeline

All data is sourced from the Binance REST API, fetched at hourly resolution over a 2 year lookback window. Five data streams are collected and merged:

- **Spot price** (`/api/v3/klines`) — open price and quote volume
- **Perpetual price and volume** (`/fapi/v1/klines`) – open price and quote volume
- **Index price** (`/fapi/v1/indexPriceKlines`) — the weighted average of spot prices across reference exchanges
- **Perpetual mark price** (`/fapi/v1/markPriceKlines`)
- **Funding rate** (`/fapi/v1/fundingRate`) — 8-hourly settlements at 00:00, 08:00, 16:00 UTC

The funding rate stream is joined to the hourly bar data using an as-of backward join (`join_asof`), which correctly propagates the most recent funding rate to each bar without look-ahead bias.

3 Feature Engineering

The feature set is constructed from the merged dataset:

Basis and basis features The basis is defined as basis $= P_{\text{perp}} - P_{\text{spot}}$, with a percentage equivalent basis_pct $=$ basis$/P_{\text{spot}}$. An 8-hour rolling mean of the basis and an 8-hour rolling standard deviation of basis_pct are also computed.

Basis Premium basis_premium $= |P_{\text{mark}} - P_{\text{index}}|/P_{\text{index}}$. This measures the magnitude of the mark price deviation from the index, used as a proxy for crossing cost in the slippage model. A signed variant, basis_premium_signed $= (P_{\text{mark}} - P_{\text{index}})/P_{\text{index}}$, is also retained to preserve directional information about whether the perp trades above or below index.

Funding features A 72-hour rolling z-score of the funding rate:

$$z_t = \frac{r_t - \mu_{72}}{\sigma_{72}}$$

where μ_{72} and σ_{72} are rolling mean and standard deviation over the prior 72 bars. A high positive z-score indicates funding is unusually elevated relative to recent history — a stronger entry signal than the raw rate alone.

Volatility Two volatility estimates: a 24-hour rolling realised volatility (annualised: $\hat{\sigma}_{24} \cdot \sqrt{24 \times 365}$) and an EWMA volatility with a 24-hour half-life. The EWMA volatility is used in the market impact model.

4 Carry Analysis

Before optimising any parameters, I first establish whether the strategy has positive expected value at all. This is done by fitting a saturation model to the average cumulative funding PnL collected after entry.

4.1 Saturation Model

For each bar where the entry signal fires (funding rate exceeds the threshold), I track the cumulative funding PnL collected over the following H hours:

$$C(h) = \sum_{t=1}^{h} \left(-\text{position} \cdot r_t^{\text{event}} \cdot N\right)$$

where r_t^{event} is zero at non-payment bars. Averaging $C(h)$ across all entries gives the expected carry curve.

The curve is fitted with a saturation model:

$$\mathbb{E}[C(h)] = A\left(1 - e^{-\lambda h}\right)$$

where A is the maximum expected carry per trade and λ is the saturation speed. The parameter A is the key pre-optimisation check: if $A <$ fees + slippage, the strategy cannot be profitable regardless of parameter choices and there is no point proceeding. For the past two years of BTC/USDT data on Binance, the fitted saturation model yields $A = \$1{,}771$ per \$100,000 notional trade. Round-trip fees alone amount to \$280 at the assumed fee rates, and slippage adds a further small cost at this notional size. Since A exceeds total execution costs by a factor of roughly six, I can comfortably say the strategy meaningfully makes pnl.

The fitted half-saturation time is $h_{1/2} = 1308.1$ hours (approximately 54.5 days), with a tight 90% bootstrapped confidence interval of [1304.6h, 1311.8h]. This means half the available carry is collected only after roughly 55 days, which directly informs the minimum viable holding period in the grid search — holding periods shorter than this leave the majority of the carry on the table. The narrow confidence interval gives high confidence in this estimate; the true half-saturation time is unlikely to deviate from 1308 hours by more than 4 hours.

5 Adverse Selection and Market Impact

A two-leg strategy trades both the perp and spot markets. Since the perp is the primary position leg, perp liquidity is the binding constraint on execution cost. I estimate Kyle lambda on the perpetual futures leg via rolling OLS:

$$\Delta P_t = \lambda \cdot Q_t^{\text{signed}} + \varepsilon_t$$

where $Q_t^{\text{signed}} = \text{sign}(\Delta P_t) \cdot V_t$ is the signed volume proxy, and the regression is estimated over a 168-bar (one-week) rolling window. Lambda measures the price impact per unit of order flow: higher lambda indicates a thinner market and more adverse selection risk.

5.1 Slippage Model

Total round-trip execution cost is modelled as:

$$\text{slippage} = \underbrace{\alpha_s \cdot s \cdot N}_{\text{spread cost}} + \underbrace{\beta \cdot \hat{\sigma} \cdot \sqrt{Q_{\text{BTC}}/V_{\text{BTC}}^{\text{hourly}}} \cdot N}_{\text{market impact}}$$

where s is the spread proxy, $\hat{\sigma}$ is EWMA volatility, Q_{BTC} is trade size in BTC, and $V_{\text{BTC}}^{\text{hourly}}$ is hourly volume in BTC. Spread cost accounts for crossing the spread and always executing at a price more unfavourable than our theo, market cost is due to the fact that putting on large trades can move the market and cause us to trade at even more unfavourable prices.

EWMA volatility is chosen over realised volatility here because it up-weights recent price action, meaning slippage estimates respond quickly to volatility regime changes rather than treating a spike two weeks ago equally to one five minutes ago.

Rather than hardcoding the impact coefficient β, it is calibrated empirically from the median Kyle lambda estimate:

$$\hat{\beta} = \frac{\hat{\lambda} \cdot \sqrt{Q_{\text{BTC}} \cdot V_{\text{BTC}}^{\text{hourly}}}}{\hat{\sigma}}$$

Applying this calibration to two years of BTC/USDT data yields impact beta = 0.00577, which is used throughout the backtest and grid search. This value implies that a $100,000 trade executed against median perp liquidity incurs a market impact cost of approximately $0.00577 \times \hat{\sigma} \times \sqrt{Q_{\text{BTC}}/V_{\text{BTC}}}$ per unit notional. This value is small but certainly worth factoring in. Calibrating empirically is sensible for this project as impact beta can vary depending on the market conditions, product and liquidity levels.

6 Backtest Engine

The full backtest iterates over hourly bars, entering a position when the funding rate breaches the threshold and exiting after a fixed holding period. Key implementation details:

- **Execution lag**: entry executes at bar $t+1$ after the signal fires at bar t, preventing look-ahead bias. This does however introduce some inaccuracy as in reality we would probably not have an hour's lag between signal generation and execution, but for the purposes of this project this is sufficient.
- **Correct quantities**: spot PnL is computed as $-\text{pos} \cdot \Delta P_{\text{spot}} \cdot (N/P_{\text{spot}}^{\text{entry}})$ and perp PnL similarly.
- **Funding accumulation**: the funding PnL loop iterates bar-by-bar between entry and exit, accumulating $-\text{pos} \cdot r_t^{\text{event}} \cdot P_t \cdot Q_{\text{BTC}}$ at each payment bar, matching the exact cash flow of a live position.
- **Non-overlapping trades**: after exit, the index advances to the later of (the bar after exit) or (entry bar + `min_trade_gap_hours`), which defaults to 8 hours — one full funding period — preventing both overlapping positions and rapid re-entry immediately after exit.

Sharpe Calculation: Sharpe is computed from a full daily PnL series spanning the entire window, with zeros on non-trading days. This correctly penalises the strategy for days where capital is deployed but no funding payment is received.

7 Strategy Optimisation

A parameter grid search sweeps over funding threshold and holding period:

Parameter	**Search range**
Funding threshold	0.00004, 0.00006, 0.00008, 0.00010, 0.00015, 0.00020, 0.00030
Holding period	14, 21, 28, 35, 42, 56, 63, 70, 77 days

The minimum holding period is informed by the alpha decay half life.

For each of the 63 combinations, the backtest produces a trade PnL series. The annualised Sharpe is computed from a daily PnL time series — each trade's PnL is assigned to its entry date, and the Sharpe is:

$$\text{Sharpe} = \frac{\bar{P}_{\text{daily}}}{\hat{\sigma}_{\text{daily}}} \cdot \sqrt{252}$$

It is worth noting that out of all the combinations there will of course be a highest one, however this doesn't necessarily mean there is alpha there. This is accounted for by our earlier carry analysis and the trade logic, which reassures us that any maximal Sharpe is sufficient reason for us to have confidence in those parameters.

7.1 Out of Sample Testing

After fitting the data and choosing parameters from the strategy optimizer, we run the backtest engine on different set of data and we do this twice. Our findings are thus:

For the first train block, ie the first 6 months of data, we find the best threshold is 0.00008, with best length of holding time being 28 days with a best sharpe of 1.804. Indeed when applied to the next 6 months to backtest these parameters we find we achieve a total pnl of roughly $521.5 and a sharpe of 0.56 with 6 trades executed in this time. This out of sample test shows the strategy is encouraging, however there is too little data as of yet to have complete faith.

For the second train block, ie the third 6 months of data, the best threshold is 0.00008, and the best holding time is again 42 days with a Sharpe of 2.208. However in this case the strategy does not work on the next 6 months as it yields a loss of -$40.8 with 3 trades being executed and a Sharpe of -0.128. Some reasons this could have occurred is a regime changes between those 6 months and the next 6 months, meaning the data we trained on is no longer representative of the market conditions.

8 Theory-to-Implementation Frictions

Trade Direction: I infer flow from the sign of the price change, but what would be better would be access to trade data where we can see executed trades and infer flow from there.

Hourly bars: I use hourly bars and, for a signal generated at time t, I execute at time t+1 which is an hour later, which is almost certainly unrealistic. Using shorter bars would make the execution costs and modelling much more accurate, however the tradeoff is that it could introduce noise and overpower the important part of the strategy, which is the funding carry, which is over a large period of time, therefore longer bars can be justified.

The carry approximation vs exact accumulation trade-off. Computing exact funding PnL requires iterating bar-by-bar between entry and exit for every trade in every grid cell. With 63 parameter combinations this is a large number of inner loop iterations in Python. The practical trade-off is speed versus accuracy: the approximation is fast enough for a grid search but the exact loop is necessary for the final reported results.

Spread Proxy: Basis premium is used as somewhat of a spread proxy however it isn't exactly this. A better estimate of spread would come from historical order book level data of the spread between the highest bid and the lowest offer.

Overlapping Trades: Due to the strict condition of non overlapping trades, if there is a signal during which a trade is on, we cannot increase the size of the trade, which perhaps in practice we would want to do.

Sharpe reliability: Since only 2 or 3 trades are executed in the 6 month periods, the Sharpe is probably not as reliable as it should be, and more data could be used to combat this, however would cause longer loading times and introduce more noise.

Conclusion

This project set out to answer a single question: is there exploitable carry alpha in BTC/USDT perpetual funding rates, and if so, under what conditions is it best captured? The saturation model gives a clear answer to the first part — with $A = \$1{,}771$ gross carry per \$100,000 notional against roughly \$280 in round-trip fees, the strategy makes meaningful pnl. The half-saturation time of 55 days confirms that carry slowly accumulates.

The out-of-sample results are mixed but informative. The first test period produces a Sharpe of 1.804 and positive PnL, suggesting the parameters generalise. The second is less convincing, which is not surprising — a strategy calibrated on six months of one regime will not always survive a regime shift. This is an inherent limitation of any carry strategy: the alpha is real, but it is regime-dependent, and knowing when the regime has changed is as important as knowing the parameters.

The modular pipeline — data fetching, feature engineering, carry analysis, adverse selection estimation, optimisation, and backtesting — is designed to generalise beyond BTC/USDT on Binance. The most natural extension is to perpetual DEXs such as Hyperliquid, where continuous funding and protocol reward structures alter the carry dynamics in ways that are worth exploring.